

Week 3 Resources: From REINFORCE to PPO and a Trading Environment

Goal of Week 3

Week 2 introduced your first learning agent: a policy network trained with REINFORCE on CartPole. You saw gradient-based policy improvement for the first time and learned to read a reward curve.

Week 3 takes two parallel steps:

1. **Understand why REINFORCE is fragile** and use a more stable algorithm (PPO) through a well-tested library.
2. **Begin designing a trading environment** — thinking carefully about state, action, and reward before writing a single line of code.

By the end of this week you should be able to:

- Explain the core limitation of REINFORCE (high variance gradient estimates).
- Use Stable-Baselines3 to train a PPO agent and interpret the output.
- Compare REINFORCE and PPO on the same task using reward curves.
- Articulate what state, action, and reward mean for a trading agent.
- Run a minimal custom Gymnasium environment you wrote yourself.

Recap: Key Ideas from Week 2

Before moving on, confirm these are solid:

- A **parameterized policy** $\pi_\theta(a \mid s)$ maps states to action probabilities via a neural network with learnable parameters θ .
- The **REINFORCE objective** is to maximize expected return $J(\theta) = \mathbb{E}_{\pi_\theta}[G]$.
- The **policy gradient** is approximated as $\nabla_\theta J(\theta) \approx \mathbb{E}[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t]$.
- In practice we *minimise* a loss $\mathcal{L}(\theta) = -\sum_t \log \pi_\theta(a_t \mid s_t) \hat{G}_t$, where $\hat{G}_t$ is a (possibly normalised) return estimate.

If any of these feel shaky, re-read the Week 2 resources before continuing.

Why REINFORCE Breaks: The Variance Problem

REINFORCE is correct in expectation but *very* noisy in practice. Each gradient estimate is computed from a single trajectory, and trajectories can vary wildly — CartPole might last 12 steps or 500 steps by chance alone.

This variance shows up as an unstable reward curve: the agent may improve for 50 episodes, then collapse, then improve again.

Two direct consequences:

- **Slow learning.** Because each gradient step is noisy, you need many more episodes to average out the noise and make consistent progress.
- **Sensitivity to hyperparameters.** A learning rate that works one run may diverge the next. The algorithm is not robust.

Baselines as a Partial Fix

Subtracting a *baseline* b from the return does not change the expected gradient but reduces its variance:

$$\nabla_{\theta} J(\theta) \approx \mathbb{E} \left[\sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b) \right].$$

A common choice is $b = \mathbb{E}[G_t]$, the mean return — which is exactly what the normalisation step in the Week 2 skeleton did. This helps, but is not sufficient to make REINFORCE reliable on harder problems.

Proximal Policy Optimisation (PPO): The Idea

PPO (Schulman et al., 2017) addresses the variance and instability problems with two key ideas.

1. Learning from Batches, Not Single Episodes

Instead of updating the policy after each episode, PPO collects N steps of experience across multiple environment interactions, then trains on *mini-batches* sampled from that buffer. This smooths out the gradient estimates and reuses data more efficiently.

2. The Clipped Surrogate Objective

The fundamental problem with gradient ascent on $J(\theta)$ is that a large update can move θ far from where the data was collected, making the gradient estimate invalid. PPO constrains the update size by clipping the probability ratio:

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}.$$

The clipped objective is:

$$\mathcal{L}^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) \hat{A}_t \right) \right],$$

where $\hat{A}_t$ is an estimate of the *advantage* at time t (how much better action a_t was compared to the average) and $\varepsilon \approx 0.2$ is a small constant.

Intuitively: if the new policy would deviate too far from the old one, the clip kicks in and stops the update. This prevents catastrophic policy collapses that plague vanilla policy gradient.

3. The Value Network

PPO trains a *value function* $V_\phi(s)$ alongside the policy. This serves as the baseline b for advantage estimation:

$$\hat{A}_t = G_t - V_\phi(s_t).$$

A learned baseline is much more accurate than the simple mean, which is why PPO's gradient estimates have lower variance than REINFORCE.

PPO in One Sentence

“Collect batches of experience, estimate advantages using a learned value function, and update the policy with gradient ascent subject to a clipping constraint that prevents large policy changes.”

Comparing REINFORCE and PPO

Property	REINFORCE	PPO
Data per update	1 episode	N steps across many episodes
Baseline	Mean return (simple)	Learned value network
Update constraint	None	Clipped probability ratio
Typical reward curve	Noisy, unstable	Smoother, more consistent
Sample efficiency	Low	Moderate
Implementation complexity	Low	Moderate (library handles it)

In this week, you will *use* PPO through Stable-Baselines3 — a well-tested library that implements it correctly. You are not expected to implement PPO from scratch. Understanding what it does conceptually (batches, clipping, value network) is the goal.

Designing a Trading Environment

This is the most important conceptual section of Week 3. Before writing trading environment code, you need to think carefully about what the three core components mean in a financial context.

State

The state should capture everything the agent needs to make a decision. For a simple single-asset trading agent, candidates include:

- Recent price returns (e.g., last k log-returns)
- Current position (long, flat, or short)
- Unrealised profit/loss

- Volume or volatility estimates
- Time-of-day or time-in-episode signals

A key design tension: more features give the agent more information, but also make learning harder and overfitting more likely. Start with the minimum that is conceptually necessary.

Action

Two common choices:

- **Discrete:** Buy / Hold / Sell. Simple, easy to implement, suitable for PPO's categorical policy.
- **Continuous:** A real number representing target portfolio fraction or order size. More expressive, but requires a Gaussian policy and more tuning.

For Week 3, discrete actions are recommended. Continuous actions will be revisited later.

Reward

Reward design is where most practitioners make mistakes. Consider these options:

1. **Step-level P&L:** Reward = change in portfolio value at each step. Pros: immediate signal. Cons: encourages churning; ignores transaction costs.
2. **Episode return:** Reward = total portfolio return at end of episode. Pros: captures long-term behaviour. Cons: very delayed signal; slow learning.
3. **Risk-adjusted return:** Reward = Sharpe ratio or similar. Pros: penalises volatility. Cons: requires careful normalisation; harder to estimate online.

For the toy environment in the assignment, step-level P&L is used for simplicity. As you move to more realistic environments in Weeks 4–5, you will revisit this choice.

Episode Structure

An episode must have a defined start and end:

- **Fixed length:** Episode runs for T timesteps (e.g., 100 steps in the toy env). Simple and uniform; easy to reason about.
- **Market-driven:** Episode = one trading day or one week of historical data. More realistic, but episode lengths vary.
- **Event-driven:** Episode ends when a stop-loss or take-profit threshold is hit. Aligned with real trading practice but complex to implement.

A Minimal Custom Gymnasium Environment

Gymnasium defines a clean interface for custom environments. Any class that inherits from `gym.Env` and implements `reset()` and `step()` will work with SB3, PPO, and any other standard RL library.

The key methods:

- `reset()` — returns an initial observation and an info dict.
- `step(action)` — returns `(observation, reward, terminated, truncated, info)`.
- `observation_space` — a `gym.spaces` object describing valid observations.
- `action_space` — a `gym.spaces` object describing valid actions.

You will implement a toy trading environment following this interface in the assignment. The price dynamics are a *random walk* — not realistic, but simple enough to verify that your environment logic is correct before adding complexity.

Connecting Back to the Five Questions

As always, ground the new ideas in the core RL framework:

- **Agent:** The policy network (REINFORCE or PPO), now also learning a value function.
- **Environment:** CartPole (for the algorithm comparison) and your toy trading env.
- **State:** CartPole observation vector; price return and current position in the trading env.
- **Actions:** Left/right for CartPole; Buy/Hold/Sell for the trading env.
- **Reward:** +1 per step alive (CartPole); step-level P&L (trading env).

Suggested External Resources (Optional)

You only need this document and the assignment to complete Week 3. If you want more context:

- Stable-Baselines3 documentation — Getting Started section.
- OpenAI Spinning Up — PPO overview (skip the derivation; read the pseudocode).
- Schulman et al. (2017), *Proximal Policy Optimization Algorithms* — read the abstract and introduction only.

Self-Check Before Moving On

You are ready for Week 4 when you can:

- Explain in words why REINFORCE has high variance and how PPO addresses it.
- Look at a REINFORCE vs. PPO reward curve and say which is which, and why.
- Describe the state, action, and reward you chose for a trading agent, and defend those choices.
- Run your toy trading environment with a random agent and interpret the output.